

Register Machine (*machine*)

Lors de la rénovation des Casemates du Bock – un réseau de fortifications souterraines à Luxembourg-Ville – des ouvriers ont déterré un ancien coffre au trésor, scellé par un mécanisme assez particulier.

L'inscription sur le couvercle l'appelle une *machine à registres*, et explique qu'elle possède 64 registres numérotés de 0 à 63, chacun pouvant contenir un entier positif ou nul. Initialement, tous les registres contiennent 0.

Un ensemble de molettes et de boutons vous permet de saisir un *programme* composé d'une séquence d'instructions. La première instruction que vous saisissez est étiquetée 0, la suivante 1, et ainsi de suite. Il existe trois types d'instructions :

- **INCREMENT** x, i — La valeur stockée dans le registre x est augmentée de 1, puis la machine traite l'instruction étiquetée i .
- **DECREMENT** x, i, j — Si la valeur stockée dans le registre x est 0, la machine traite l'instruction étiquetée i . Sinon, le registre x est diminué de 1, puis la machine traite l'instruction étiquetée j .
- **HALT** — La machine arrête de traiter les instructions.

La machine commence le traitement à partir de l'instruction étiquetée 0. Une fois qu'elle s'arrête, elle ouvrira le coffre si la valeur stockée dans le registre 0 est exactement N .

Si elle ne s'arrête pas après avoir traité 50 000 000 instructions, ou si elle tente de traiter une instruction qui n'existe pas, la machine va au contraire tomber en panne de manière catastrophique et sceller le coffre à jamais.

Votre tâche est de saisir un programme qui ouvrira le coffre. Comme la machine est délicate, vous voulez que la longueur du programme soit la plus courte possible, et certainement pas plus de 100 instructions.

Implémentation

Vous devrez soumettre un seul fichier source `.cpp`.



Parmi les pièces jointes de cette tâche, vous trouverez un modèle `machine.cpp` et un évaluateur d'exemple `grader.cpp` que vous pouvez utiliser pour tester vos solutions localement.

Vous devrez implémenter la fonction suivante :

C++

```
void program_machine(int N)
```

- L'entier N représente le nombre à stocker dans le registre 0 lorsque la machine s'arrête.

Votre programme peut appeler les fonctions suivantes au plus 100 fois, lesquelles sont déjà définies dans l'évaluateur :

C++

```
void add_increment(int x, int i)
```

C++

```
void add_decrement(int x, int i, int j)
```

C++

void add_halt()

Évaluateur

Parmi les pièces jointes de ce problème, vous trouverez une version simplifiée de l'évaluateur (grader) utilisé lors de l'évaluation, que vous pouvez utiliser pour tester vos solutions localement. L'évaluateur d'exemple lit les données depuis `stdin`, appelle la fonction `program_machine` et écrit en retour sur `stdout` en utilisant le format suivant.

Le fichier d'entrée se compose d'une ligne, contenant l'entier N .

La sortie est composée d'une ligne :

- Si votre programme ouvre le coffre correctement, la sortie est `OK L`, où L est la longueur de votre programme.
- Sinon, la sortie commence par `WA`, suivi d'une brève raison (par exemple, le programme a dépassé 100 instructions, le programme ne s'est pas arrêté à temps, etc.)

Contraintes

- $1 \leq N \leq 1\,000\,000$.

Score

Votre programme sera testé contre plusieurs cas de test regroupés en sous-tâches. Afin d'obtenir le score d'une sous-tâche, votre programme doit ouvrir correctement le coffre pour chaque test de cette sous-tâche.

Pour la sous-tâche 3, le score est déterminé par la longueur de votre programme. Soit L la longueur maximale de votre programme sur tous les cas de test de cette sous-tâche. Le score pour cette sous-tâche est $\min(77, 100 - L)$. En particulier, pour obtenir la totalité des 77 points pour cette sous-tâche, votre programme doit utiliser au plus 23 instructions sur tous les cas de test de cette sous-tâche.

- **Sous-tâche 0 [0 points]**: Exemples.
- **Sous-tâche 1 [7 points]**: $N \leq 99$.
- **Sous-tâche 2 [16 points]**: N est une puissance de 2 (c'est-à-dire $N = 2^k$ pour un certain entier $k \geq 0$).
- **Sous-tâche 3 [77 points]**: Aucune contrainte supplémentaire. (Notez que le score pour cette sous-tâche dépend de la longueur de votre programme.)

Exemples

Évaluateur

```
program_machine(3)
```

Solution

```
add_increment(1, 1)
add_increment(0, 2)
add_decrement(1, 4, 1)
add_halt()
add_increment(0, 3)
```

Évaluateur

Solution

```
program_machine(5)
```

```
add_increment(0, 1)
add_increment(0, 2)
add_increment(0, 3)
add_increment(0, 4)
add_increment(0, 5)
add_halt()
```

Explication

Dans le premier exemple, $N = 3$. L'exemple crée un programme avec 5 instructions :

0.	INCREMENT 1, 1
1.	INCREMENT 0, 2
2.	DECREMENT 1, 4, 1
3.	HALT
4.	INCREMENT 0, 3

Le tableau suivant montre l'étiquette de chaque instruction traitée par la machine et le contenu des registres 0 et 1 après coup :

Instruction	Registre 0	Registre 1
—	0	0
0	0	1
1	1	1
2	1	0
1	2	0
2	2	0
4	3	0
3	3	0

Dans le second exemple, $N = 5$ et l'exemple de programme présenté utilise 6 instructions.